

.csv file, you can download it from the URL given at the end of this article.

```
#Create the training data
no = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,
15,16,17,18,19,20]
area = [3400,4500,6500,7700,6600,3500,4
300,3700,8700,9000,9100,9200,8200,8800,
8400,8300,7900,7400,7100,6400]
price = [1500,2300,2777,3567,2799,1633,
2111,1700,4333,4500,4551,4560,4200,4304
,4000,4230,3700,3300,3000,2600]
data = pd.DataFrame({'No':no, 'Area':are
a, 'Price':price})

data.head()
```

The `data.head()` method will help us see the first five rows of our data frame, as shown in Figure 2. Now, let us create a sample test data to test our model.

```
#create the test data
index = [1,2,3]
area_size = [1200, 1350, 5400]
area_value = [571, 643, 2570]
test_data = pd.DataFrame({'No':index,
'Area': area_size, 'Expected Value':
area_value})

test_data.head()
```

Figure 3 shows the visual of the test data. The next step is to convert the data into a proper Numpy array, which we can then use to train our model.

```
#convert the data into a numpy array
data = np.array(data,dtype=np.float32)
x_train = data[:,1:2]
y_train = data[:,2:3]

test_data = np.array(test_
data,dtype=np.float32)
x_test = test_data[:,1:2]
y_test = test_data[:, 2:3]

#visualize the data using a scatter
plot
plt.plot(x_train, y_train, 'bo')
plt.xlabel("Price ($) of land")
plt.ylabel("Size of Land")
```

	No	Area	Price
0	1	3400	1500
1	2	4500	2300
2	3	6500	2777
3	4	7700	3567
4	5	6600	2799

Figure 2: The top five rows of the training data

```
plt.title("Land purchase trends")
plt
```

Figure 4 shows the visualisation of the data. From the scatter plot, we can clearly see a positive correlation in the data, i.e., as the size of the land increases, so does its price. Hence, we can draw a line across this data to derive predictions of what the value of a given piece of land might be, given its size.

We will define a linear regression class with the help of the `torch.nn` module, which has a linear model available that takes two parameters — input size and the output size. Additionally, we will also define a forward class method inside our function to derive the output of the model.

```
class LinReg(torch.nn.Module):
    def __init__(self, input_size,
```

	No	Area	Expected	Value
0	1	1200		571
1	2	1350		643
2	3	5400		2570

Figure 3: Test data

```
output_size):
    super(LinReg, self).__init__()
    self.linear = torch.
nn.Linear(input_size, output_size)

    def forward(self, x):
        out = self.linear(x)
        return out
```

We can instantiate the model using the following code:

```
#define input/output dimensions, learning
rate and the no. of epochs to run
input_dimension = 1
output_dimension = 1
learn_rate =0.000000001
epochs = 100

model = LinReg(input_dimension, output_
dimension)
##### For GPU #####
if torch.cuda.is_available():
    model.cuda()
```

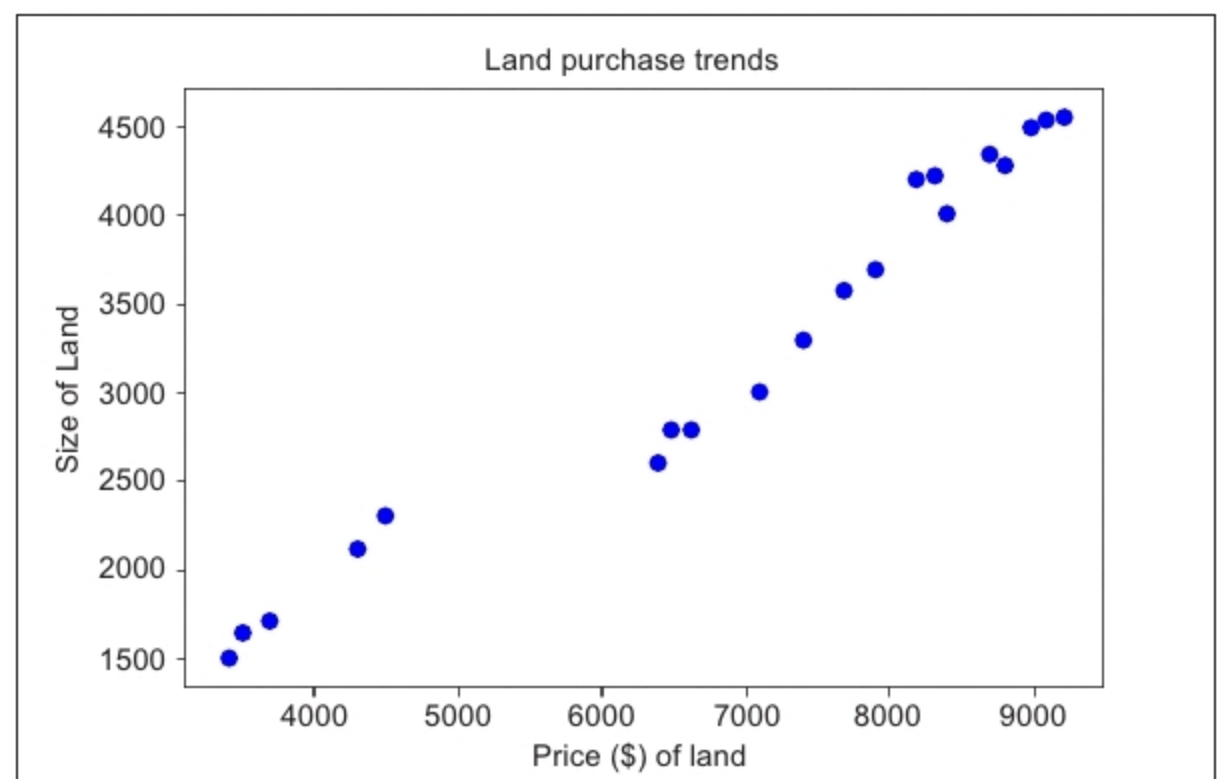


Figure 4: Visualising the data using a scatter plot